# Behaviour-Driven Testing — Group / Ungroup figures

**Feature:** Group / Ungroup figures (JHotDraw 7) **Automation:** JGiven (BDD) + AssertJ (assertions) | **Module:** `L9/bdd-tests`

## User story

> **As a** user creating a diagram in the drawing editor, **I want to** group several selected figures into a single object (and ungroup it again later), **so that** I can move, resize and arrange related shapes together as one unit.

## User story → BDD Given-When-Then scenarios

| Scenario | Given | When | Then |
|---|---|---|---|
| **Group figures** | a drawing with figures *rectangle, ellipse, line* | I group the selected figures | the drawing contains a single group **and** the group contains 3 figures |
| **Ungroup a group** | a drawing containing a group of *rectangle, ellipse* | I ungroup the group | the drawing contains the figures *rectangle, ellipse* **and** contains no group **and** the released figures are *rectangle, ellipse* |
| **Empty selection (boundary)** | an empty drawing | I group the selected figures | the group is empty |

## Automating the scenarios with JGiven

The scenarios are written as executable JGiven scenarios. JGiven splits each scenario into three reusable **stage** classes whose method names read as the sentence fragments above:

- `GivenDrawing` — `a_drawing_with_figures(...)`, `a_drawing_containing_a_group_of(...)`, `an_empty_drawing()`
- `WhenUser` — `I_group_the_selected_figures()`, `I_ungroup_the_group()`
- `ThenOutcome` — `the_drawing_contains_a_single_group()`, `the_group_contains_$_figures(n)`, `the_drawing_contains_no_group()`, …

State is shared between stages with `@ProvidedScenarioState` / `@ExpectedScenarioState`, and the test class wires them together:

```
public class GroupUngroupScenarioTest
        extends ScenarioTest<GivenDrawing, WhenUser, ThenOutcome> {

    @Test
```

```
    public void selected_figures_can_be_grouped_into_a_single_object() {
        given().a_drawing_with_figures("rectangle", "ellipse", "line");
        when().I_group_the_selected_figures();
        then().the_drawing_contains_a_single_group()
              .and().the_group_contains_$_figures(3);
    }
}
```

JGiven produces a human-readable report of each scenario (under `target/jgiven-reports`) in the same Given/When/Then language as the table above, so the tests double as living documentation of the feature.

## Domain-specific assertions with AssertJ

The `Then` stage uses AssertJ for fluent, domain-readable assertions, e.g.:

```
assertThat(drawing.figures()).hasSize(1);
assertThat(drawing.figures().get(0)).isInstanceOf(FigureGroup.class);
assertThat(released).containsExactly("rectangle", "ellipse");
assertThat(drawing.figures()).noneMatch(f -> f instanceof FigureGroup);
```

> The lab notes that **AssertJ-Swing** would be used to drive the scenarios for a live Swing UI. Because the JHotDraw 7 build is not runnable here, the scenarios are automated against the same Group/Ungroup domain logic used in the Testing Lab (with a concrete in-memory `Drawing`), and AssertJ-core provides the domain assertions. The Given-When-Then structure is identical to what an AssertJ-Swing driver would express against the GUI.

## Verification

The scenarios run automatically in the **CI pipeline** (`.github/workflows/ci.yml`, step "*Build and test L9 bdd-tests*") on every push and pull request. A green run is the evidence that the feature behaves as the user story specifies.